

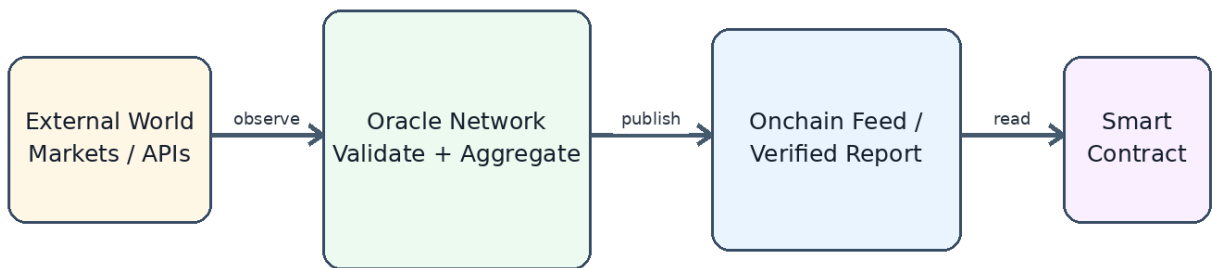
DAY 16

Oracles & Blockchain Automation

Data Feeds, Secure Price Consumers & Chainlink CRE

Complete 60-Minute Beginner-to-Freelancer Lecture

Hybrid Smart Contract: Onchain Logic + Offchain Data



Goal: Build the mental model and code patterns needed to consume trustworthy external data and automate blockchain workflows safely.

2026 course update: Chainlink Data Feeds remain the core price-data lesson. For automation and custom offchain workflows, this lecture teaches Chainlink Runtime Environment (CRE) as the current path and treats the older Automation / Functions interfaces only as migration knowledge.

1. Learning Objectives and 60-Minute Teaching Plan

By the end of Day 16, a beginner should be able to explain the oracle problem, consume a Chainlink Data Feed through AggregatorV3Interface, validate price freshness and decimals, add application-level bounds, understand L2 sequencer safeguards, describe CRE triggers and workflow execution, and scope an oracle/automation task for a client.

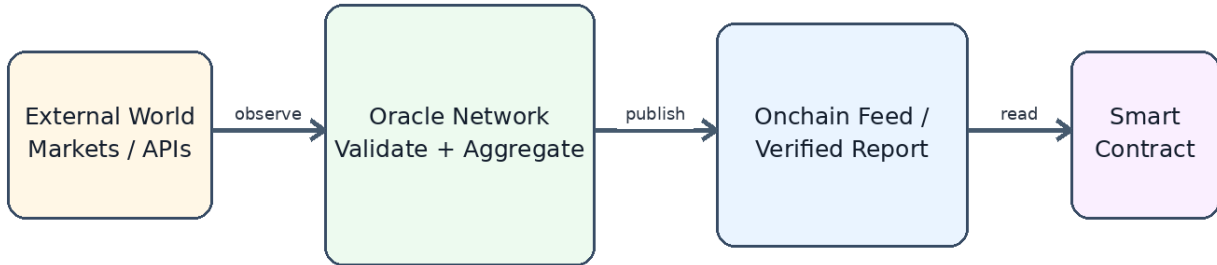
Time	Topic	Outcome
0-5 min	Oracle problem	Explain why deterministic smart contracts need external-data bridges.
5-12 min	Oracle trust model	Understand source quality, aggregation, freshness and application controls.
12-22 min	Chainlink Data Feeds	Trace source -> DON -> aggregator -> proxy -> consumer.
22-32 min	Safe Solidity consumer	Read latestRoundData, validate answer, timestamp, decimals and bounds.
32-36 min	L2 safeguards	Use sequencer uptime status and grace-period reasoning.
36-43 min	2026 automation landscape	Understand CRE and migration from legacy Automation / Functions.
43-51 min	CRE workflows	Learn Cron, HTTP and EVM-log triggers plus HTTP/EVM capabilities.
51-56 min	Practical lab	Build PriceGuard and design a CRE monitoring workflow.
56-59 min	Client/security scenarios	Turn requirements into controls, tests and deliverables.
59-60 min	Review	Recite the oracle validation checklist.

60-second target explanation: A blockchain oracle does not make offchain information inherently true; it is infrastructure for obtaining, validating, aggregating and delivering data. A safe application still chooses the correct feed, checks freshness and decimals, handles network-specific risks, defines acceptable bounds, and decides what should happen when data is unavailable.

2. The Oracle Problem: Why Smart Contracts Need External Data

EVM contracts execute deterministically from onchain state and transaction inputs. They cannot safely call an arbitrary website during Solidity execution and trust the response. DeFi, insurance, prediction markets, tokenized assets and automated operations therefore need a controlled bridge between onchain logic and offchain information.

Hybrid Smart Contract: Onchain Logic + Offchain Data



Use case	Offchain / external information needed	Onchain action
Lending	Collateral price	Borrow limit / liquidation check
Stablecoin	Reference price or reserve data	Mint/redeem risk controls
Prediction market	Verified outcome	Settlement
Tokenized asset	NAV / reserves / rate data	Valuation or minting logic
Automation	Time, API result, event or state condition	Execute scheduled/conditional transaction

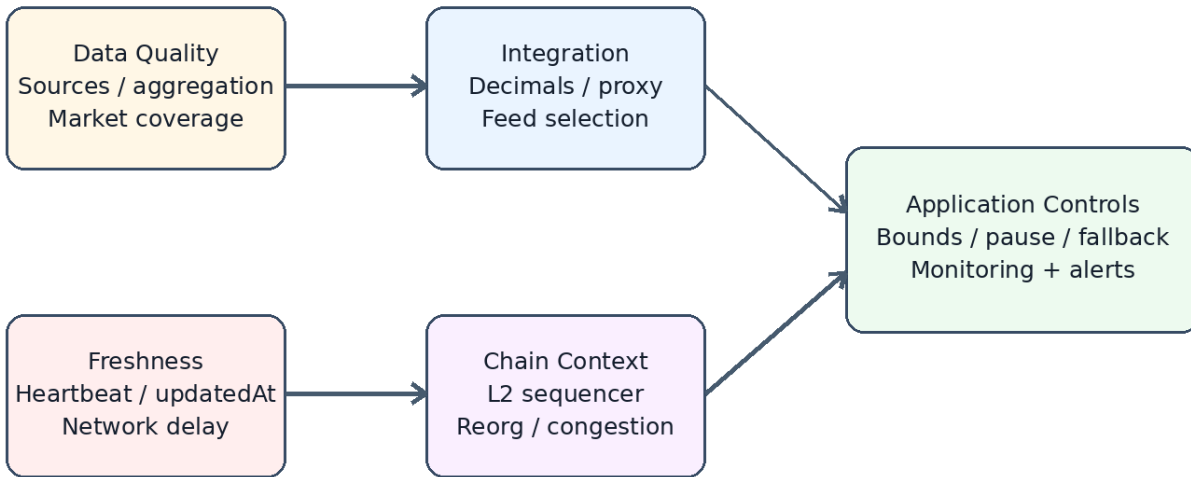
2.1 Oracle != API endpoint

- An API is a data source/interface.
- An oracle system is the mechanism that observes data, applies rules/aggregation and makes a value usable by blockchain logic.
- A single API + single signer creates a different trust model from a decentralized oracle network.
- The application developer remains responsible for how the delivered value is validated and used.

3. Oracle Trust Model: What Can Go Wrong?

A professional oracle review separates four questions: Is the source appropriate? Is the value fresh? Is the integration correct? What happens if the data is missing or abnormal?

Oracle Security Is Layered Risk Management



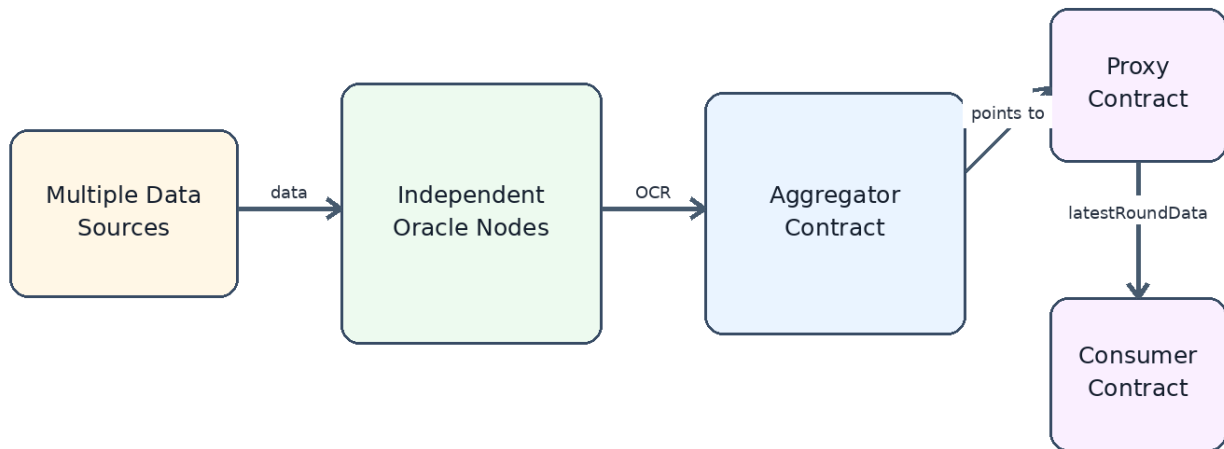
Risk	Example	Developer control
Wrong feed	Using BTC/USD when logic expects WBTC/USD	Verify feed description, network and official address catalog
Stale value	Feed has not updated within the application tolerance	Check updatedAt against maxAge
Bad unit assumption	Treating 8-decimal answer as 18 decimals	Read decimals() and normalize explicitly
Extreme market event	Price is technically fresh but outside intended operating region	Application-level min/max, deviation or pause logic
L2 sequencer outage	Users cannot interact normally while price-sensitive actions continue	Sequencer uptime feed + grace period
Admin/integration error	Feed address changed to a malicious contract	Immutable/config-governed addresses, multisig/timelock, monitoring

Core principle: Do not ask only "Is the oracle secure?" Ask "Is this oracle configuration safe for this specific business rule, chain, asset and failure mode?"

4. Chainlink Data Feeds: Architecture

For single-value EVM Data Feeds, the consumer normally reads a proxy contract using `AggregatorV3Interface`. The proxy points to an underlying aggregator, allowing the feed implementation to be updated without forcing every consumer to change its integration.

Chainlink Data Feed Architecture



Component	Responsibility
Data sources	Provide market/reference observations.
Oracle nodes / DON	Independently observe and participate in offchain reporting/consensus.
Aggregator	Stores aggregated onchain answer and round metadata.
Proxy	Stable consumer-facing address that points to the current aggregator.
Consumer	Your smart contract or offchain application reading the feed.

Best practice: Integrate through the feed proxy using `AggregatorV3Interface` rather than binding application logic directly to a specific underlying aggregator implementation.

5. AggregatorV3Interface: The Functions You Must Know

```
import {AggregatorV3Interface} from
"@chainlink/contracts/src/v0.8/shared/interfaces/AggregatorV3Interface.sol";

AggregatorV3Interface public immutable priceFeed;
```

Function	Purpose
decimals()	Number of decimals in the returned answer.
description()	Human-readable feed description.
version()	Proxy/aggregator version information.
latestRoundData()	Latest answer plus timestamps and round metadata.
getRoundData(roundId)	Historical round lookup where supported/appropriate.

5.1 latestRoundData() return values

```
(
    uint80 roundId,
    int256 answer,
    uint256 startedAt,
    uint256 updatedAt,
    uint80 answeredInRound
) = priceFeed.latestRoundData();
```

Value	Meaning for application code
answer	The feed value; validate sign/range before casting to uint256.
updatedAt	Timestamp of the latest update; use it for freshness policy.
roundId	Round identifier; useful for diagnostics/logging.
startedAt	When the round started; also used by sequencer uptime feeds.
answeredInRound	Legacy/deprecated metadata; do not build new safety logic around it.

```
uint8 d = priceFeed.decimals();
(, int256 answer,, uint256 updatedAt,) = priceFeed.latestRoundData();

// Human value = answer / 10**d
```

[illegible]

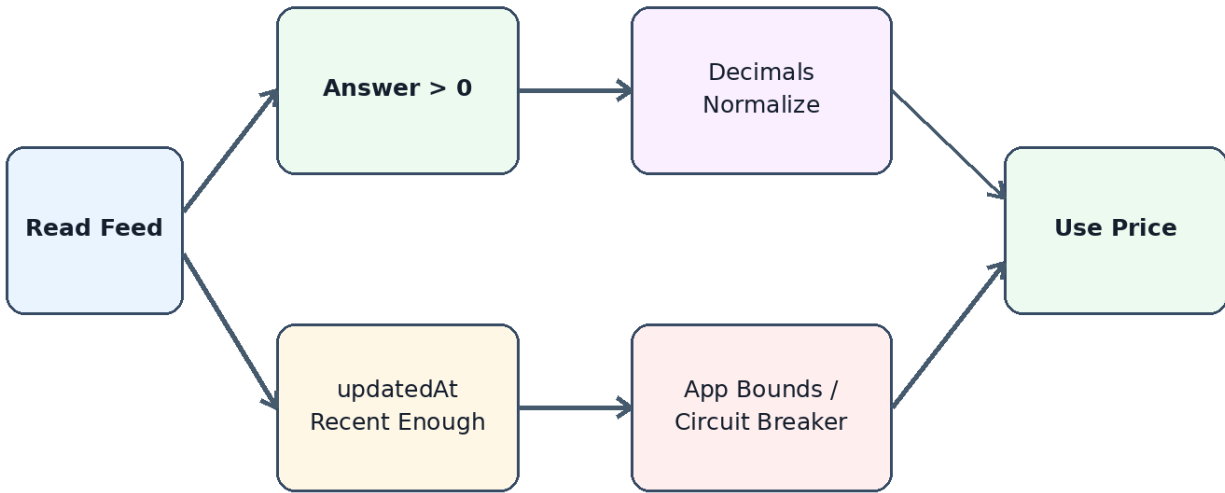
```
function scaleTo1e18(uint256 value, uint8 sourceDecimals)
    internal pure returns (uint256)
{
    if (sourceDecimals == 18) return value;
    if (sourceDecimals < 18) return value * 10 ** (18 - sourceDecimals);
    return value / 10 ** (sourceDecimals - 18);
}
```

Day 16 - Oracles & Blockchain Automation | 7

7. Freshness: updatedAt Is Part of the Price

Data Feeds update according to feed-specific deviation and heartbeat rules rather than providing a continuous stream. Your application must define how old is too old for its risk model.

Safe Price Consumer Validation Pipeline



```

if (updatedAt == 0 || updatedAt > block.timestamp) {
    revert InvalidTimestamp();
}
if (block.timestamp - updatedAt > maxAge) {
    revert StalePrice();
}

```

7.1 maxAge is an application policy

- Do not blindly choose 1 hour, 1 day or another constant because an example used it.
- Check the feed heartbeat/deviation configuration and the business consequences of stale data.
- A lending liquidation path may need stricter behavior than a read-only analytics dashboard.
- When data is too old, decide whether to revert, pause risky actions, switch to a restricted mode, or use a carefully designed fallback.

8. Application-Level Bounds and Circuit Breakers

A fresh value can still be unsafe for a particular application. Chainlink documentation recommends application-specific monitoring and reasonable limits because aggregator min/max values are not a substitute for your own risk controls on most feeds.

```
if (answer <= 0) revert InvalidPrice();
if (answer < minAnswer || answer > maxAnswer) {
    revert PriceOutsideBounds();
}
```

Control	What it protects against	Design question
Positive-value check	Invalid negative/zero price	Can this feed legitimately return negative values?
Min/max bounds	Absurd or unsupported operating region	Who can update bounds and with what governance delay?
Deviation check	Sudden jump relative to previous trusted state	How much movement is normal for this asset/time window?
Pause mode	Continuing dangerous operations during data incident	Which actions remain available while paused?
Monitoring alerts	Silent degradation	Who receives alerts and what is the response runbook?

Important: A circuit breaker is business logic, not a universal constant. Overly tight limits can freeze legitimate markets; overly loose limits can fail to reduce risk.

9. Complete Solidity Example: OraclePriceGuard

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

import {AggregatorV3Interface} from
    "@chainlink/contracts/src/v0.8/shared/interfaces/AggregatorV3Interface.sol";

contract OraclePriceGuard {
    AggregatorV3Interface public immutable feed;
    uint256 public immutable maxAge;
    int256 public immutable minAnswer;
    int256 public immutable maxAnswer;

    error InvalidPrice();
    error InvalidTimestamp();
    error StalePrice(uint256 age);
    error PriceOutsideBounds(int256 answer);

    constructor(address feed_, uint256 maxAge_, int256 min_, int256 max_) {
        require(feed_ != address(0), "feed=0");
        require(maxAge_ > 0, "maxAge=0");
        require(min_ < max_, "bad bounds");
        feed = AggregatorV3Interface(feed_);
        maxAge = maxAge_;
        minAnswer = min_;
        maxAnswer = max_;
    }

    function latestPrice()
        public view returns (uint256 price, uint256 updatedAt)
    {
        (, int256 answer, , uint256 ts,) = feed.latestRoundData();
        if (answer <= 0) revert InvalidPrice();
        if (ts == 0 || ts > block.timestamp) revert InvalidTimestamp();

        uint256 age = block.timestamp - ts;
        if (age > maxAge) revert StalePrice(age);
        if (answer < minAnswer || answer > maxAnswer)
            revert PriceOutsideBounds(answer);

        return (uint256(answer), ts);
    }

    function feedDecimals() external view returns (uint8) {
        return feed.decimals();
    }
}
```

10. Review the PriceGuard Like a Freelancer

The code is intentionally simple enough for a beginner, but the review questions are production-oriented.

Review question	Why it matters
Is feed the correct proxy on this exact chain?	Wrong network/feed address can return unrelated data or fail.
Does maxAge match feed characteristics and protocol risk?	Freshness policy must be contextual.
Are min/max values expressed in the feed raw decimals?	Unit mismatch creates false reverts or false acceptance.
Does the protocol need an L2 sequencer check?	Price-sensitive L2 actions can be unfair/unsafe around downtime.
Can the app tolerate a revert when the feed is unavailable?	Failure mode must be designed, not discovered in production.
Should parameters be immutable or governance-controlled?	Operational flexibility changes admin risk.
Are monitoring and incident procedures defined?	Contract checks alone do not provide operational awareness.

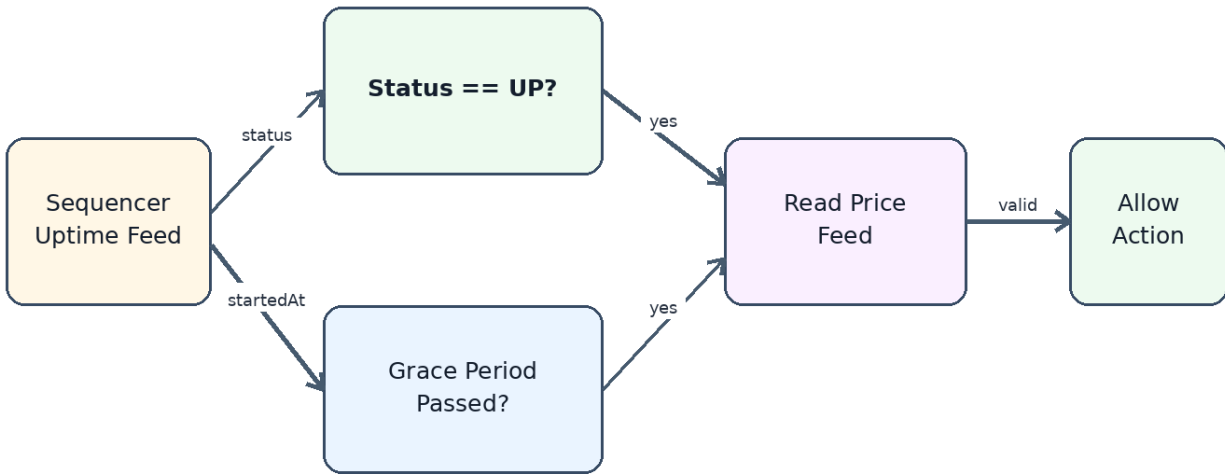
10.1 Client deliverable language

Instead of saying "I integrated Chainlink," describe the actual controls: official proxy address verification, decimals normalization, staleness threshold, application bounds, L2 sequencer handling where applicable, Foundry tests, deployment configuration and monitoring assumptions.

11. L2 Sequencer Uptime Feeds

On many L2 systems, a sequencer outage changes users' ability to interact normally. Price-sensitive applications should not treat the situation as identical to normal operation. Chainlink provides sequencer uptime feeds for supported L2 networks.

L2 Price Safety: Sequencer Status + Grace Period



```

(
  , int256 answer, uint256 startedAt,
) = sequencerUptimeFeed.latestRoundData();

bool isSequencerUp = answer == 0;
if (!isSequencerUp) revert SequencerDown();

uint256 timeSinceUp = block.timestamp - startedAt;
if (timeSinceUp <= GRACE_PERIOD) revert GracePeriodNotOver();

```

- Status 0 means up; status 1 means down in the documented uptime-feed model.
- After recovery, a grace period gives users time to regain normal access before sensitive operations resume.
- The main price feed still needs its own freshness and value validation after the sequencer checks pass.

12. Derived Prices and Multiple Feeds

Sometimes the required pair is not a direct feed. A developer may derive A/B using A/USD and B/USD, but combining feeds introduces multiple decimal systems, freshness windows and error margins.

```
// Conceptual only: both values first normalized to the same scale.
```

```
priceAperB = priceAUSD * SCALE / priceBUSD;
```

Concern	Example failure
Mismatched decimals	A/USD uses one decimal format, B/USD another.
Different update times	A feed is fresh while B feed is near/over its tolerance.
Division rounding	Integer division truncates and may bias small values.
Different market assumptions	Wrapped, staked or bridged asset may not perfectly track underlying.
Overflow/precision	Multiplication before division may exceed safe range or lose precision.

Rule: Validate each source feed independently before combining them, normalize units explicitly, then test boundary values and rounding behavior.

13. Testing Oracle Consumers

Oracle consumers should be tested with controlled mock responses. Do not write tests that depend only on whatever a public testnet feed happens to return today.

Test	Expected result
Positive fresh answer	Returns value and timestamp.
Zero answer	Reverts InvalidPrice.
Negative answer	Reverts InvalidPrice for price use case.
updatedAt == 0	Reverts InvalidTimestamp.
updatedAt in future	Reverts InvalidTimestamp.
Age exactly maxAge	Define and test boundary behavior.
Age > maxAge	Reverts StalePrice.
Below min / above max	Reverts PriceOutsideBounds.
Decimals query	Matches configured mock feed.
L2 sequencer down	Sensitive operation reverts.
Grace period not passed	Sensitive operation reverts.
Recovery + valid price	Operation succeeds.

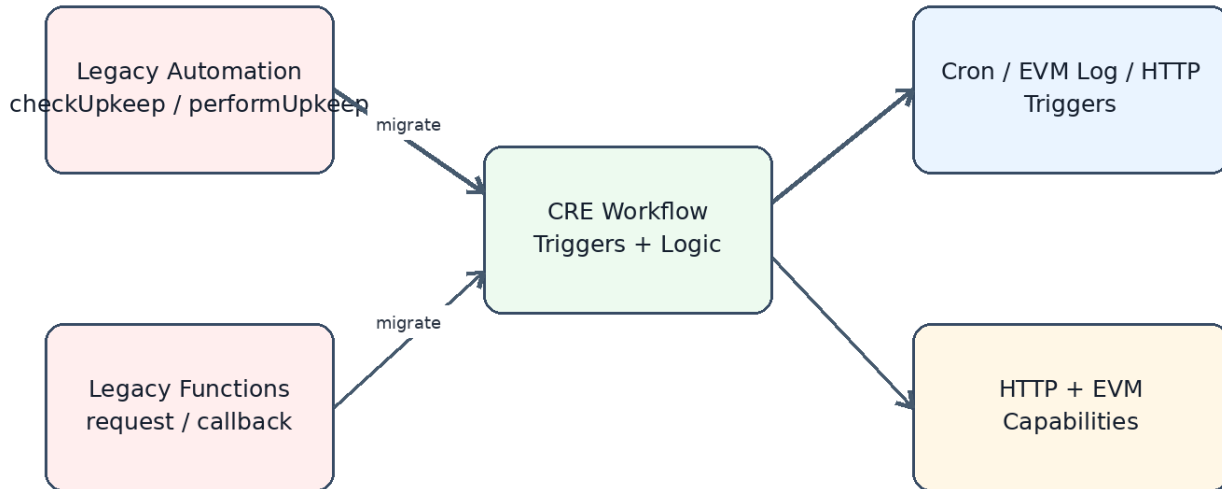
13.1 Foundry test skeleton

```
function test_RevertWhenPriceIsStale() public {
    mock.setRoundData(3000e8, block.timestamp - MAX_AGE - 1);
    vm.expectRevert();
    guard.latestPrice();
}
```

14. 2026 Update: Automation and Custom Offchain Compute

This course originally planned a classic Chainlink Automation + Functions lesson. The 2026 platform path changed, so a job-ready lecture must teach the current architecture instead of training you primarily on interfaces that production users were told to migrate away from.

2026 Automation / Functions Migration Mental Model



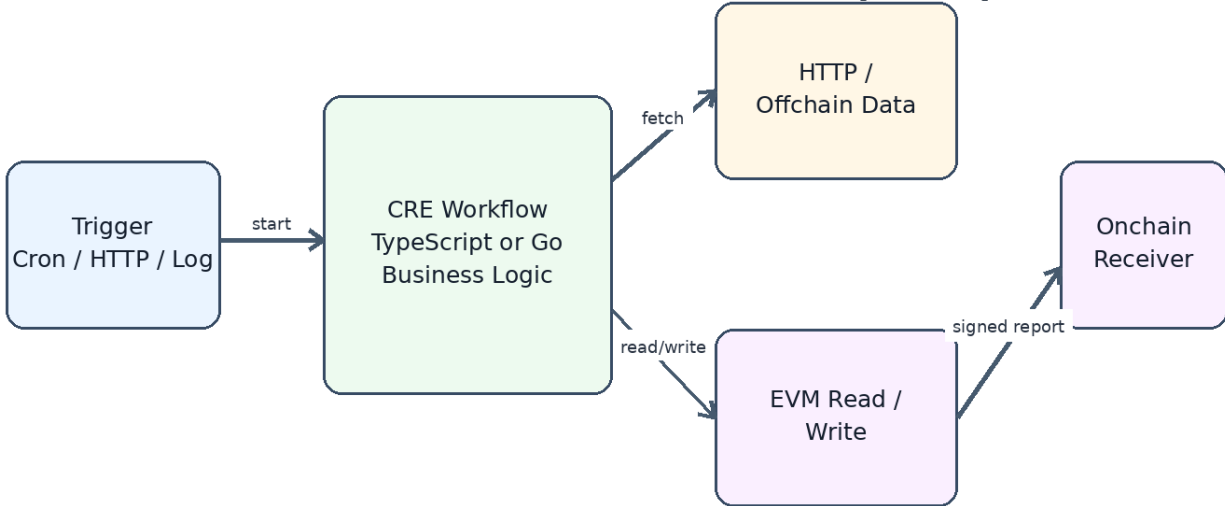
Older pattern	Current learning target
Automation checkUpkeep() / performUpkeep()	Understand for legacy maintenance; design new orchestration with CRE workflows.
Functions request + fulfill callback	Understand when reading old code; use CRE HTTP/offchain capabilities for new workflow design.
Time-based upkeep	CRE Cron trigger.
Log-triggered upkeep	CRE EVM Log trigger.
Offchain check + onchain action	Workflow handler performs reads/logic and writes a signed report/onchain action.

Date-sensitive note: Chainlink documentation states Functions mainnet sunset on June 30, 2026 and Automation v2.1 sunset on July 31, 2026. As of this Day 16 lecture (August 2026), CRE is the relevant new-development orchestration layer.

15. Chainlink Runtime Environment (CRE): Mental Model

CRE workflows are written using the TypeScript or Go SDK, compiled and simulated through the CRE tooling, then run across a decentralized oracle network when deployed. Workflows can react to triggers and use capabilities such as HTTP and EVM interactions.

Chainlink CRE Workflow Architecture (2026)

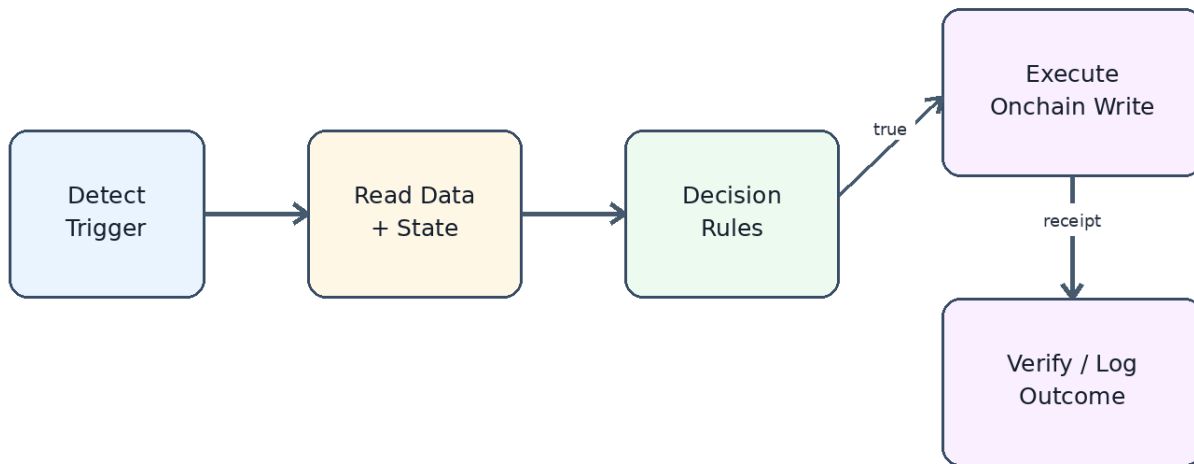


CRE concept	Beginner meaning
Workflow	Offchain business logic executed by CRE.
Trigger	What starts a run: Cron, HTTP request or EVM log.
HTTP capability	Fetch external API/data as part of workflow logic.
EVM client	Read from or write to EVM chains.
Workflow DON	Coordinates workflow execution.
Capability DON	Performs specialized tasks such as data or chain interaction.
Consensus	Independent nodes execute/verify and aggregate results.
Simulation	Compile/run workflow locally for debugging before production deployment.

16. CRE Triggers: When Should Automation Run?

Trigger	Use when	Freelancer example
Cron	A schedule should initiate work	Every hour verify reserve ratio and publish status.
EVM Log	A contract event should initiate work	SettlementRequested event starts offchain outcome lookup.
HTTP	An authorized external request should initiate work	Operations system requests a verified onchain update.

Automation Pattern: Detect -> Decide -> Execute



16.1 Separate trigger from decision

A trigger should not automatically imply that an expensive or risky onchain transaction must happen. A workflow can start, read state/data, evaluate conditions, then skip the write if the action is unnecessary.

```

trigger
-> read external/onchain data
-> validate and normalize
-> evaluate business condition
-> if false: log/exit
-> if true: create verified report / onchain write
-> verify result
  
```

17. CRE Workflow Pseudocode: Price Sentinel

CRE SDK details evolve, so the lecture focuses on the stable architecture. Use the current official template/SDK reference when implementing the exact imports and deployment commands.

```
// PSEUDOCODE - architecture, not copy/paste SDK code
workflow on Cron("*/10 * * * *") {
  onchain = evm.read(priceGuard.latestPrice())
  apiRisk = http.get("https://example-risk-api/asset/ETH")

  result = consensus({ onchain, apiRisk })

  if (result.riskScore >= LIMIT) {
    report = encodeAlert(result)
    evm.write(receiver.onReport(report))
  }
}
```

17.1 Production questions

- What network/chain selector and receiver address are used?
- What exact trigger frequency is needed and what is the cost/operational quota?
- Does the HTTP source require secrets?
- What data must reach consensus before writing onchain?
- Who is authorized to accept the signed report?
- Can the same report be replayed?
- How is idempotency handled if execution is retried?
- What monitoring/logging proves a run succeeded or failed?

18. Onchain Receiver Pattern

A CRE workflow that writes verified data onchain commonly targets a receiver/bridge pattern. The receiving contract must authenticate the expected forwarder/report path before applying state changes.

```
contract RiskReceiver {
    address public immutable trustedForwarder;
    uint256 public lastRiskScore;
    uint256 public lastUpdatedAt;

    error Unauthorized();

    constructor(address forwarder) {
        trustedForwarder = forwarder;
    }

    function receiveVerifiedReport(bytes calldata report) external {
        if (msg.sender != trustedForwarder) revert Unauthorized();

        (uint256 score, uint256 observedAt) =
            abi.decode(report, (uint256, uint256));

        require(observedAt <= block.timestamp, "future timestamp");
        lastRiskScore = score;
        lastUpdatedAt = observedAt;
    }
}
```

Security note: This simplified receiver demonstrates authorization and payload validation, not the full current CRE receiver interface. For production, use the current official receiver template and report-verification pattern rather than inventing your own protocol.

19. Traditional Backend vs Data Feed vs CRE

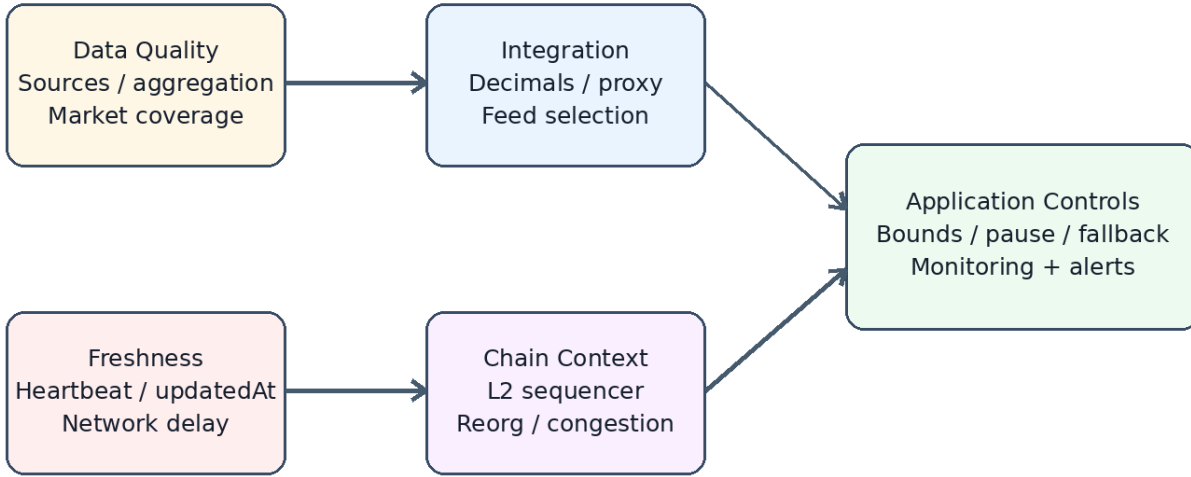
Not every automation task needs decentralized execution. A freelancer should recommend the simplest architecture that meets the client's trust and availability requirements.

Option	Best fit	Tradeoff
Direct Data Feed read	Contract needs standardized onchain data such as price	Simple and composable; limited to published feed data.
Normal backend worker	Client controls infrastructure and only needs operational automation	Cheap/flexible, but backend operator is trusted.
CRE workflow	Client needs trigger + offchain logic/data + verifiable/decentralized execution model	More platform-specific setup and operational design.
User-triggered contract	Action can safely happen only when a user calls	No automation infra, but depends on user activity.
Protocol incentive model	Third parties are rewarded to call a permissionless function	Decentralized participation, but incentive/game design is required.

Freelancer value: Clients pay for architecture judgment, not for forcing every problem into the newest tool. Explain why the trust model justifies the chosen solution.

20. Security Checklist for Oracle-Driven Automation

Oracle Security Is Layered Risk Management



Layer	Checklist
Feed identity	Correct chain, proxy, asset pair, description and official address source.
Value validation	Sign, decimals, bounds, precision and derived-price math.
Freshness	updatedAt / maxAge and feed heartbeat/deviation understanding.
L2	Sequencer status and recovery grace period if applicable.
Automation auth	Only trusted/verified receiver path can perform privileged writes.
Replay/idempotency	Repeated report or retry does not duplicate irreversible action.
Failure mode	Define revert, pause, restricted mode or fallback behavior.
Admin security	Multisig/timelock for mutable critical config when needed.
Monitoring	Alerts on stale feeds, abnormal values, failed runs and config changes.
Testing	Mocks, boundary tests, fork/integration tests and incident scenarios.

21. Hands-On Lab: PriceGuard + Automated Monitoring Design

The lab can be completed on a local chain or testnet. Use test assets only and verify current official feed addresses before deployment.

1. Create a Foundry project and install/import the Chainlink contracts package/interface.
2. Deploy OraclePriceGuard with a test/mock feed first.
3. Test positive price, stale timestamp, zero/negative answer and min/max bounds.
4. Replace the mock with a verified testnet Data Feed proxy configuration.
5. Read feedDecimals(), description() and latestPrice().
6. If targeting a supported L2, add a sequencer uptime guard and grace-period tests.
7. Design a CRE Cron workflow that reads the guard every 10 minutes.
8. Add a decision rule: only emit/write an alert if price or freshness condition crosses a threshold.
9. Define receiver authorization, replay/idempotency and monitoring behavior.
10. Write a one-page client handoff: addresses, assumptions, thresholds, tests, deployment commands and incident behavior.

21.1 Expected portfolio architecture

```
contracts/  
  OraclePriceGuard.sol  
  RiskReceiver.sol  
  
test/  
  OraclePriceGuard.t.sol  
  OraclePriceGuardL2.t.sol  
  
workflow/  
  price-sentinel/      # CRE TS/Go project or design notes  
  
docs/  
  architecture.md  
  deployment.md  
  security.md
```

22. Upwork / LinkedIn Client Scenarios

Client request	Your first technical response
"Add Chainlink ETH/USD to my lending contract."	Confirm chain, official feed proxy, decimals, heartbeat tolerance, liquidation behavior, L2 sequencer needs and tests before editing liquidation math.
"Run a function every hour."	Ask what condition actually requires a transaction; evaluate CRE Cron vs normal backend vs permissionless incentive model.
"Fetch weather/API data into Solidity."	Explain that arbitrary HTTP is offchain; design a CRE workflow/verified report rather than direct Solidity HTTP.
"My oracle value is wrong by 10 billion."	Inspect decimals/unit normalization and pair/feed identity first.
"Price is fresh but protocol should not trade during an extreme move."	Add application risk bounds/deviation/pause logic, not just staleness.
"Our old Chainlink Automation upkeep stopped."	Identify registry/version and migration status; migrate logic to current CRE architecture rather than building new dependency on sunset service.
"Our old Functions request no longer works."	Treat as legacy migration: map request/callback logic into a CRE trigger + HTTP/offchain workflow + verified onchain receiver.

23. Debugging Matrix

Symptom	Likely causes	Checks
latestRoundData reverts	Wrong address, unsupported feed/network, RPC issue	Explorer code, official feed catalog, chain ID
Price too large/small	Decimals mismatch	feed.decimals(), normalization math
StalePrice reverts often	maxAge too strict or feed heartbeat misunderstood	updatedAt, feed configuration, network congestion
Works on L1, unsafe on L2	Missing sequencer guard	Uptime feed integration and grace period
Automation does nothing	Trigger/config/deploy access/receiver auth/condition false	Workflow logs, simulation, trigger, receiver, decision rule
Duplicate actions	Retry/replay not idempotent	Report IDs/nonces/state guard
Receiver rejects report	Wrong forwarder/authorization/payload schema	Current CRE template/config, sender, ABI decode
Unexpected pause during volatility	Bounds too tight	Business thresholds and monitoring data

23.1 Debug in this order

1. Verify chain ID and contract addresses.
2. Read raw feed response and decimals.
3. Check timestamps and application thresholds.
4. Check L2 sequencer state if relevant.
5. Reproduce with a mock or fork test.
6. Inspect workflow simulation/logs.
7. Inspect receiver authorization and emitted events.
8. Only then modify business logic.

24. Interview Questions - Practice Out Loud

Q1. What is the blockchain oracle problem?

Smart contracts cannot safely trust arbitrary offchain data directly; an oracle system bridges external information into a form onchain logic can consume under an explicit trust model.

Q2. Why use a Data Feed proxy?

The proxy gives consumers a stable interface/address while the underlying aggregator can be upgraded.

Q3. What does latestRoundData return?

Round ID, answer, startedAt, updatedAt and legacy answeredInRound metadata.

Q4. Why check updatedAt?

To enforce an application-specific freshness requirement.

Q5. Why read decimals()?

Feed answers are integers whose human meaning depends on the feed decimal scale.

Q6. What is an L2 sequencer uptime feed for?

To detect sequencer downtime and support a recovery grace period before resuming sensitive operations.

Q7. What is CRE?

Chainlink Runtime Environment is the current workflow/orchestration layer for trigger-based offchain/onchain logic using TypeScript or Go workflows run across DONs.

Q8. Name three CRE triggers.

Cron, HTTP and EVM log.

Q9. Why is "fresh oracle" not enough security?

The value may still be wrong for the intended pair/unit or outside the application risk envelope.

Q10. When would you choose a normal backend instead of CRE?

When centralized operational trust is acceptable and decentralized/verifiable orchestration adds unnecessary complexity.

25. Day 16 Quiz

1. A Data Feed consumer should normally call the proxy or the underlying aggregator directly?
2. Which latestRoundData field is most useful for freshness validation?
3. Why must you read decimals()?
4. What should happen if a price answer is zero for a normal asset-price feed?
5. What extra feed can protect price-sensitive L2 applications around sequencer downtime?
6. Name the three CRE trigger types taught today.
7. What older Chainlink service used checkUpkeep() and performUpkeep()?
8. What current architecture replaces new use of Chainlink Functions after its 2026 sunset?
9. Why should automation logic be idempotent?
10. Give two application-level oracle controls beyond checking updatedAt.
11. Should maxAge be copied blindly from an example? Why?
12. What is the first thing to verify when an oracle integration behaves differently on another chain?

Answer Key

1. Proxy through AggregatorV3Interface.
2. updatedAt.
3. To interpret/normalize the integer answer correctly.
4. Reject/revert unless the specific feed semantics legitimately allow it.
5. L2 Sequencer Uptime Feed plus a recovery grace period.
6. Cron, HTTP, EVM Log.
7. Chainlink Automation.
8. Chainlink Runtime Environment (CRE) workflow + current receiver/report pattern.
9. Retries/replays must not duplicate irreversible state changes.
10. Examples: min/max bounds, deviation checks, pause/restricted mode, L2 sequencer guard.
11. No; it must match feed behavior and application risk tolerance.
12. Chain ID/network plus the exact official feed/receiver addresses and configuration.

26. Homework and Portfolio Assignment

Homework A - Oracle consumer

- Implement OraclePriceGuard with custom errors and NatSpec.
- Add a scaleTo1e18 helper and unit tests for feed decimals below/equal/above 18.
- Write mock-feed tests for zero, negative, stale, future timestamp and boundary prices.

Homework B - L2 safety

- Add a sequencer uptime check to a separate L2PriceGuard version.
- Test sequencer down, grace period active and normal recovery.

Homework C - Automation architecture

- Design a CRE workflow for one use case: reserve alert, scheduled snapshot, event-driven settlement request or risk monitor.
- Document trigger, external/onchain reads, decision logic, receiver write, authorization, idempotency and failure behavior.
- Use the current CRE simulator/templates when implementing exact SDK code; record any platform/version assumptions in README.

Portfolio deliverable

Repository title: Oracle Price Guard + Automated Risk Sentinel. Include architecture diagram, contracts, Foundry tests, workflow design/code, deployment configuration, security assumptions, screenshots/logs from simulation/testnet, and a client-facing handoff document.

27. Day 16 Cheat Sheet

Topic	Remember
Oracle problem	Offchain facts require an explicit bridge/trust model.
Data Feed path	Sources -> DON -> aggregator -> proxy -> consumer.
Integration interface	AggregatorV3Interface against the proxy.
Core read	latestRoundData().
Freshness	updatedAt + application maxAge.
Units	answer + decimals belong together.
Bounds	Use application-specific reasonable limits/circuit breakers.
L2	Sequencer uptime status + recovery grace period.
CRE	Current workflow orchestration using TypeScript/Go SDK + CLI.
CRE triggers	Cron, HTTP, EVM Log.
Legacy knowledge	Automation/Functions are migration-maintenance topics in 2026.
Automation safety	Authenticate receiver, make writes idempotent, monitor failures.
Freelancer workflow	Confirm chain/feed -> define risk policy -> code -> mocks/tests -> deploy -> monitor -> document.

The 10-second oracle checklist

CORRECT FEED?
 CORRECT CHAIN?
 CORRECT DECIMALS?
 ANSWER VALID?
 TIMESTAMP FRESH?
 L2 SEQUENCER SAFE?
 APP BOUNDS OK?
 FAILURE MODE DEFINED?
 AUTOMATION AUTHENTICATED?
 TESTS + MONITORING READY?

28. Primary Technical References and 2026 Platform Notes

Use primary documentation when implementing because feed addresses, supported networks, SDKs, templates and platform migration guidance can change.

Chainlink Data Feeds: <https://docs.chain.link/data-feeds>

Data Feeds API Reference: <https://docs.chain.link/data-feeds/api-reference>

Using Data Feeds on EVM Chains: <https://docs.chain.link/data-feeds/using-data-feeds>

L2 Sequencer Uptime Feeds: <https://docs.chain.link/data-feeds/l2-sequencer-feeds>

Chainlink Runtime Environment (CRE): <https://docs.chain.link/cre>

Migrate Chainlink Automation to CRE: <https://docs.chain.link/cre/reference/cla-migration-go>

Migrate Chainlink Functions to CRE: <https://docs.chain.link/cre/reference/clf-migration-ts>

CRE Supported Networks: <https://docs.chain.link/cre/supported-networks-ts>

OpenZeppelin Contracts 5.x: <https://docs.openzeppelin.com/contracts/5.x>

Version note: This lecture is intentionally dated August 2026 for the automation sections. Re-check current Chainlink CRE deployment access, SDK/CLI versions, templates and supported networks before using exact production commands.

Preparation for Day 17

Day 17 moves from single-chain EVM development to multi-chain thinking: Ethereum, BNB Chain, Base, Arbitrum, Polygon and Solana concepts, deployment differences, RPC/chain configuration, bridge assumptions, token addresses and how to scope multi-chain client work.

29. Instructor Delivery Notes - 60-Minute Speaking Script

Minute	Instructor prompt
0-5	Ask: "Can Solidity call a weather API directly during execution?" Use the answer to introduce deterministic execution and the oracle problem.
5-12	Draw four risk boxes: source, freshness, integration, application response. Emphasize that oracle security is contextual.
12-22	Draw Data Feed architecture. Show proxy vs aggregator and read latestRoundData/decimals.
22-32	Live-code PriceGuard. Pause at answer sign, updatedAt and decimals. Ask students to predict each failure mode.
32-36	Explain sequencer outage unfairness on L2 and the grace-period concept.
36-43	Explain the 2026 platform migration: legacy Automation/Functions knowledge is useful for maintenance, CRE is the current workflow lesson.
43-51	Draw trigger -> workflow -> HTTP/EVM -> receiver. Give one Cron and one EVM-log example.
51-56	Assign the Price Sentinel lab and ask students to identify trust boundaries and idempotency requirements.
56-59	Read two client scenarios and have students answer with clarification questions rather than code.
59-60	Students recite: feed, chain, decimals, answer, freshness, L2, bounds, failure mode, auth, tests/monitoring.

Day 16 completion standard: A student is ready to continue when they can integrate a price feed without unit/staleness mistakes, explain L2 sequencer protection, and describe a current CRE workflow architecture without confusing it with the sunset Automation/Functions model.